

Day 44



getters, setters & instanceof Operator in JavaScript:

Getters in JavaScript

A getter is a special method that lets you access a property like a variable, but behind the scenes a function runs. Used to return computed or formatted values.

Syntax

```
class User {  
  constructor(name) {  
    this._name = name; // "_" used to show it's private-like  
  }  
  
  get name() { // getter method  
    return this._name.toUpperCase();  
  }  
}
```

```
const u = new User("aditya");  
console.log(u.name); // ADITYA (accessed like a property)
```

Key Points

- Accessed like a property: obj.property
- No need to call like a function.
- Good for computed values and hiding internal logic.

Example:

```
JS main.js > ...  
1 class Person {  
2   constructor(name) {  
3     this._name = name; // private-like variable  
4   }  
5   // Getter  
6   get name() {  
7     return this._name;  
8   }  
9   // Setter  
10  set name(value) {  
11    this._name = value;  
12  }  
13 }  
14 const p = new Person("Alice");  
15 console.log(p.name); // uses getter + "Alice"  
16 p.name = "Bob"; // uses setter  
17 console.log(p.name); // "Bob"  
...  
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS  
● PS E:\JavaScript\Day41-50\Day44> node main.js  
Alice  
Bob
```

This code creates a Person class where the name is stored in a private-like variable `_name`. The getter `name()` lets you read the person's name, and the setter `name(value)` lets you change it safely. When you create a new person (`new Person("Alice")`), the getter returns "Alice". Then the setter updates the name to "Bob", and the getter shows the updated value.

Setters in JavaScript

A setter allows you to set/update a property using simple assignment (`obj.property = value`). Used for validation, formatting, or controlling updates.

Syntax

```
class User {
  constructor(name) {
    this._name = name;
  }

  set name(value) {      // setter method
    if (value.length < 3) {
      console.log("Name too short!");
      return;
    }
    this._name = value;
  }
}

const u = new User("Adi");
u.name = "Ak";    // too short → validation triggers
u.name = "Akshay"; // setter updates value
```

Key Points

- Used like a normal assignment.
- Good for validation & controlling updates.
- Usually paired with a getter.

instanceof Operator:

Checks if an object belongs to a specific class. Returns true/false.

Example:



```
JS instanceof.js > ...
1  class Person {
2    constructor(name) {
3      this._name = name;
4    }
5    get name() {
6      return this._name;
7    }
8    set name(value) {
9      this._name = value;
10   }
11 }
12 const p = new Person("Alice");
13 // Using instanceof
14 console.log(p instanceof Person); // true
15 console.log(p instanceof Object); // true
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS E:\JavaScript\Day41-50\Day44> node .\instanceof.js
true
true

`instanceof` checks whether an object was created from a certain class.

Since `p` was created using `new Person()`, `p instanceof Person` is true. And because all classes in JavaScript come from the built-in `Object`, `p instanceof Object` is also true.